AGENTIC AI SUMMIT

David Parry

Developer Advocate
Qodo

Hands-On Lab: Building AI-Powered Agents for Your IDE Plugin

Training | Beginner

Virtual | July 16-31

qodo

🚀 What You'll Build: MCP Server + AI Agent Integration

- **Build a complete MCP server** from scratch in Java
  - JSON-RPC transport layer
  - Protocol handshake & message routing
  - Resources, Tools & Prompts capabilities
- **Create powerful AI Agent analysis tool that work with MCP server Tool**
  - Keyword search across codebases
  - Structured data extraction
  - Seamless agent integration
- **Bridge deterministic tools with intelligent agents**
  - MCP tools handle the "what" (data collection)
  - AI agents provide the "why" (insights & context)
  - Together: Raw data → Actionable intelligence

qodo

📅 Workshop Journey: 5 Hands-On Sessions

```
git clone git@github.com:David-Parry/agent-mcp-workshop.git
```

- Chapter 1 → Transport Layer Foundation
- Chapter 2 → JSON-RPC Protocol Deep Dive
- Chapter 3 → Protocol Handshake Implementation
- Chapter 4 → Resources, Tools & Prompts
- Chapter 5 → Agent Configuration & Integration

qodo

Chapter 1

Transport Layer Foundation

```
git checkout 01-chapter
```

- **MCP Transport Foundation**: Introduces the core transport layer for bidirectional JSON message communication between MCP servers and clients using event-driven architecture.
- **Three Key Components**: IOHandler manages thread-safe I/O operations, LogFile provides process-specific logging, and Router defines the extension point for message handling.
- **Robust Design**: Emphasizes reliability through error handling, thread safety, graceful degradation, and comprehensive logging for debugging distributed systems.

Chapter 2

JSON-RPC Protocol Deep Dive

```
git checkout 02-chapter
```

- **JSON-RPC 2.0 Foundation: MCP builds on JSON-RPC 2.0 specification for structured message exchange, following a stateless request-response pattern with support for both synchronous requests and asynchronous notifications.**
- **Protocol Architecture: Features transport-agnostic design (works over STDIO, SSE, HTTP), bidirectional communication enabling real-time updates, and extensibility through capability-based feature detection and optional parameters.**
- **Structured Data Model: Implements resources with URIs and MIME types, tools with JSON Schema definitions, and prompts with argument specifications for standardized data exchange between clients and servers.**

Chapter 3

Protocol Handshake Implementation

```
git checkout 03-chapter
```

- **MCP Protocol Handshake: Implements the critical initialization flow where clients and servers exchange capabilities through initialize requests/responses, establishing protocol version and supported features for subsequent communication.**
- **Core Message Routing: Builds the foundational routing infrastructure using IORouter with pattern matching to handle different JSON-RPC message types (requests, responses, notifications) and implements basic handlers for ping and cancellation notifications.**
- **Type-Safe Communication: Establishes proper deserialization infrastructure with typed parameters instead of raw JSON, emphasizing STDIO discipline for protocol communication and testing with MCP Inspector to validate the implementation.**

Chapter 4

Resources, Tools & Prompts

```
git checkout 04-chapter
```

- **Resources Implementation: Adds content exposure capability using JavadocResources helper to serve HTML documentation from classpath, implementing RESOURCES_LIST for discovery and RESOURCES_READ for retrieval with proper MIME types.**
- **Tools Implementation: Creates executable functionality through KeyWordSearch tool with JSON Schema-defined parameters, implementing TOOLS_LIST for discovery and TOOLS_CALL for execution with comprehensive error handling.**
- **Prompts Implementation: Builds user-friendly templates that guide tool usage through PROMPTS_LIST handler, creating a "search_keyword" prompt that bridges user intent with tool execution through intelligent client-side autocomplete.**

Chapter 5

Agent Configuration & Integration

```
git checkout 05-chapter
```

- **Configuration Architecture: Establishes the connection between AI agents and MCP servers through two key files - mcp.json defining server launch parameters and agent.toml configuring agent access to tools and AI models.**
- **Division of Responsibilities: MCP tools handle deterministic operations (file traversal, pattern counting, data aggregation) while AI agents provide intelligent interpretation, context analysis, and actionable insights from the raw data.**
- **Agent-Tool Integration: Demonstrates the complete workflow where user commands trigger agents that leverage MCP tools for data collection, then apply AI reasoning to transform raw frequency data into meaningful contextual insights about code patterns.**

qodo

Thanks!

qodo